

1.4 Das Entwurfsmuster Kompositum

Die Liste aus 1.3 funktioniert. Aber fast jede Methode muss Sonderfälle abfangen: das Ende der Liste (nachfolger == null) und die leere Liste (anfang == null). *Website: Station 1.4, Projekt Kompositum*

Aufgabe 1) Fallunterscheidungen zählen

a) Zähle in deiner Lösung von 1.3 die Abfragen auf null. b) Stell dir vor, am Ende der Liste stünde statt null ein echtes Objekt, der **Abschluss**. Was müsste er bei den einzelnen Methoden antworten?

Methode	== null in Liste	== null in Knoten	Antwort des Abschlusses
laenge()	1	1	0: Ab hier gibt es keinen Knoten mehr.
enthaelt(d)	1	1	false: Hier ist nichts mehr zu finden.
add(d)	1	1	Er erzeugt den neuen Knoten (Genaueres in Aufgabe 4).
entfernen(d)	1	1	Nichts gefunden: Es bleibt, wie es ist.

Mit Abschluss gibt es nie mehr null: Die leere Liste besteht nur aus dem Abschluss.

Knoten und Abschluss sollen beide als nachfolger und als anfang stehen können. Sie brauchen also einen gemeinsamen Typ:

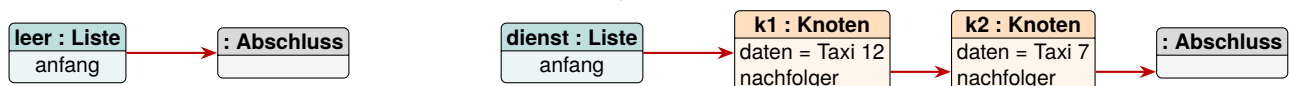
```
public abstract class ListElement {
    public abstract int laenge();
    public abstract boolean enthaelt(DatenElement d);
    public abstract ListElement add(DatenElement d);
    public abstract ListElement entfernen(DatenElement d);
    ...
}
public class Abschluss extends ListElement { ... }
public class Knoten extends ListElement {
    private DatenElement daten;
    private ListElement nachfolger;
    ...
}
```

Abstrakte Klasse und Kompositum

- Von einer **abstrakten Klasse** kann man keine **Objekte** erzeugen. Ihre abstrakten Methoden haben keinen **Rumpf**: Jede Unterklasse muss sie **überschreiben**.
- Anders als ein Interface darf eine abstrakte Klasse auch **Attribute** und fertige Methoden enthalten. Erben kann eine Klasse aber nur von **einer** Klasse.
- Welche `laenge()` läuft, entscheidet das Objekt zur Laufzeit: **Polymorphie**.
- Kompositum**: Statt Fallunterscheidungen gibt es für jeden Fall eine eigene **Klasse**.

Aufgabe 2) Objektdiagramm

Zeichne alle Verweise ein: links eine leere Liste, rechts eine Liste mit Taxi 12 und Taxi 7.



Aufgabe 3) Projekt Kompositum, TODO 3a und 3b

```
// Knoten
public int laenge() {
    return nachfolger.laenge() + 1;
}
public boolean enthaelt(DatenElement d) {
    if (daten.istGleich(d)) {
        return true;
    } else {
        return nachfolger.enthaelt(d);
    }
}
// Abschluss
public int laenge() { return 0; }
public boolean enthaelt(DatenElement d) { return false; }
```

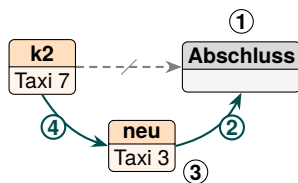
a) Ergänze und übertrage ins Projekt.

b) In `Knoten.laenge()` steht keine Abbruchbedingung mehr. Warum endet die Rekursion trotzdem?

Am Ende ruft der letzte Knoten `laenge()` des Abschlusses auf. Der antwortet 0 und ruft niemanden mehr auf.

Aufgabe 4) Projekt Kompositum, TODO 4a bis 4c

Beim Anhängen gibt es ein Problem: Der Abschluss kann den nachfolger seines Vorgängers nicht ändern. Lösung: Jedes add gibt zurück, **was danach an seiner Stelle steht**. Der Aufrufer trägt diese Rückgabe ein.



- ① Der Abschluss erzeugt einen neuen Knoten,
- ② trägt sich selbst als dessen Nachfolger ein
- ③ und gibt den neuen Knoten zurück.
- ④ k2 trägt die Rückgabe als Nachfolger ein und gibt sich selbst zurück.

```

// Abschluss (4a)
public ListElement add(DatenElement d) {
    Knoten neu = new Knoten(d); // ①
    neu.setNachfolger(this); // ②
    return neu; // ③
}

// Knoten (4b)
public ListElement add(DatenElement d) {
    nachfolger = nachfolger.add(d); // ④
    return this;
}

// Liste (4c)
public boolean add(DatenElement d) {
    anfang = anfang.add(d);
    return true;
}
  
```

b) Warum muss auch die Liste das Ergebnis speichern (anfang = ...)? Denke an die leere Liste!
In der leeren Liste ist anfang der Abschluss. Nach add muss anfang auf den neuen Knoten zeigen.

Aufgabe 5) Projekt Kompositum, TODO 5

```

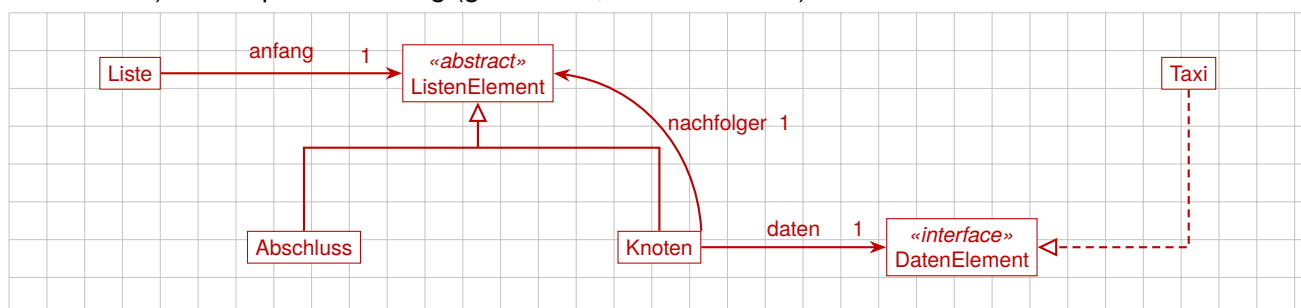
// Knoten: Rückgabe = wer danach hier steht
public ListElement entfernen(DatenElement d) {
    if (daten.istGleich(d)) {
        return nachfolger; // ich werde übersprungen
    } else {
        nachfolger = nachfolger.entfernen(d);
        return this;
    }
}

// Abschluss
public ListElement entfernen(DatenElement d) {
    return this;
}
  
```

- a) Ergänze Knoten.entfernen. Warum ist Abschluss.entfernen schon fertig?
Kommt die Suche beim Abschluss an, gab es d nicht: Alles bleibt, wie es ist.
- b) Vergleiche mit 1.3. Was fällt weg?
die Vorausschau auf den Nachfolger und der Sonderfall für den ersten Knoten

Aufgabe 6) Klassendiagramm

Zeichne das Klassendiagramm (nur Klassennamen) mit Liste, ListElement, Knoten, Abschluss, DatenElement und Taxi: Assoziationen mit Namen und Multiplizitäten, Vererbung (durchgezogen, leeres Dreieck) und Implementierung (gestrichelt, leeres Dreieck).



Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- ListElement könnte auch ein Interface sein. Was spricht hier dafür, was dagegen?
- Jede Liste hat ihren eigenen Abschluss. Würde auch **ein** gemeinsamer Abschluss für alle Listen reichen?
- Knoten.add gibt this zurück, obwohl sich an seiner Stelle nichts ändert. Warum trotzdem?